

Universidad de San Carlos de Guatemala

Facultad de ingeniería

Primer Semestre Vacaciones 2026

Laboratorio Organización Computacional

Sección A

Ing. Carlos Lozano

Aux. Carlos Blanco



Estudiante:

Carnét:

Jose Daniel Salguero Saldivar

202444320

Adrián Obed Coronado Bueso

202444480

Eric Esteban Ramos Sagastume

202445229

Randy Xavier Lone Peñate

202341660

Angie Renata Godoy Portillo

202441178

Kristhoper Franklin Ucelo Pirir

202502428

Introducción:

La lógica digital fue una pieza fundamental para este proyecto, ya que permitió procesar y transformar información usando solo ceros y unos. Al ver el funcionamiento de una ALU tradicional, se diseñó y construyó LogicCalc, una calculadora digital basada en lógica combinacional que permitió llevar la teoría del curso de Organización Computacional a la práctica real. El sistema fue diseñado para recibir dos números de 4 bits y, mediante señales de control, permitió elegir entre un bloque aritmético que sumaba, restaba, multiplicaba, calculaba potencias y comparaba magnitudes, y un bloque lógico encargado de las operaciones AND, OR, NAND y XNOR.

Para esta práctica, se utilizaron componentes electrónicos como compuertas básicas, multiplexores, sumadores y decodificadores, permitiendo mostrar los resultados en displays de 7 segmentos. Además, se diseñaron los módulos del circuito, se simuló su funcionamiento en Proteus y se realizó el montaje físico en protoboard o PCB, logrando comprender de forma práctica cómo se integran distintos componentes lógicos para formar un sistema funcional.

Objetivos:

General

Diseñar e implementar una calculadora digital basada en lógica combinacional, capaz de realizar operaciones aritméticas, lógicas y comparativas.

Específicos

Construir los módulos aritmético, lógico y comparativo utilizando compuertas lógicas e integrados combinacionales.

Simular el funcionamiento del circuito en Proteus para verificar que las operaciones seleccionadas generen los resultados correctos.

Mostrar los resultados del sistema mediante displays de 7 segmentos y LEDs, según el tipo de operación ejecutada.

Funciones Booleanas

Las funciones booleanas permiten representar el comportamiento lógico de los módulos del circuito LogicCalc. En este proyecto se aplicaron para describir el controlador, las operaciones lógicas, la unidad aritmética, la potencia, la multiplicación y el comparador.

El sistema trabaja con dos operandos binarios de 4 bits:

$$A = A_3 A_2 A_1 A_0$$

$$B = B_3 B_2 B_1 B_0$$

También se utilizan tres señales de selección:

S2, S1 y S0

Estas señales permiten elegir la operación que se desea ejecutar dentro del circuito. Para las ecuaciones, el símbolo ' representa negación, el símbolo $\cdot$ representa AND, el símbolo + representa OR y el símbolo $\oplus$ representa XOR.

Controlador de operaciones

El controlador activa el módulo correspondiente según la combinación de las señales S2, S1 y S0.

$$\text{Suma} = S_2' \cdot S_1' \cdot S_0'$$

$$\text{Resta} = S_2' \cdot S_1' \cdot S_0$$

$$\text{Multiplicación} = S_2' \cdot S_1 \cdot S_0'$$

$$\text{Potencia} = S_2' \cdot S_1 \cdot S_0$$

$$\text{AND} = S_2 \cdot S_1' \cdot S_0'$$

$$\text{OR} = S_2 \cdot S_1' \cdot S_0$$

$$\text{NAND} = S_2 \cdot S_1 \cdot S_0'$$

$$\text{XNOR} = S_2 \cdot S_1 \cdot S_0$$

Estas combinaciones permiten que solamente una operación sea seleccionada a la vez.

Unidad lógica

La unidad lógica trabaja bit a bit entre los operandos A y B. Para cada posición de bit se generan las siguientes salidas:

$$\text{AND} = A \cdot B$$

$$\text{OR} = A + B$$

$$\text{NAND} = (A \cdot B)'$$

$$\text{XNOR} = (A \oplus B)'$$

Como los operandos son de 4 bits, estas operaciones se repiten para cada bit: A0 con B0, A1 con B1, A2 con B2 y A3 con B3. El resultado se muestra mediante LEDs.

Unidad de suma

La suma binaria se basa en el comportamiento de un sumador completo. Cada bit del resultado depende del bit de A, del bit de B y del acarreo de entrada.

$$S = A \oplus B \oplus C$$

$$C_{out} = A \cdot B + C \cdot (A \oplus B)$$

Donde S representa el bit de suma, C representa el acarreo de entrada y Cout representa el acarreo de salida.

Unidad de resta

La resta se realiza utilizando complemento a dos, por lo que el operando B se invierte y se suma una unidad.

$$A - B = A + B' + 1$$

Esto permite utilizar una estructura similar a la suma, pero modificando el sustraendo. De esta forma, el circuito puede obtener el resultado de $A - B$ usando lógica combinacional.

Unidad de multiplicación

La multiplicación se obtiene mediante productos parciales. Cada producto parcial se genera con una operación AND entre bits de A y B.

$$\text{Producto parcial} = \text{bit de A} \cdot \text{bit de B}$$

Luego, estos productos parciales se suman para obtener el resultado final. Este principio permite implementar una multiplicación binaria usando compuertas AND y sumadores.

Unidad de potencia

La potencia se desarrolla reutilizando la lógica de multiplicación. Para calcular el cuadrado, el número se multiplica por sí mismo:

$$A^2 = A \cdot A$$

Para calcular el cubo, el resultado anterior se vuelve a multiplicar por A:

$$A^3 = A \cdot A \cdot A$$

Por eso, el módulo de potencia se considera una extensión del bloque multiplicador.

Unidad comparativa

La unidad comparativa determina la relación entre los operandos A y B. Las salidas principales del comparador son:

$$A > B$$

$$A < B$$

$$A = B$$

Para verificar igualdad, se comparan los bits correspondientes de ambos operandos. Si todos los bits son iguales, entonces $A = B$.

Cuando A y B son diferentes, el circuito compara desde el bit más significativo hasta el menos significativo para determinar cuál número es mayor y cuál es menor. El resultado se muestra en displays de 7 segmentos.

Activación de salidas

Las operaciones aritméticas y comparativas se visualizan en displays de 7 segmentos. Las operaciones lógicas se visualizan mediante LEDs de 4 bits.

$$\text{Unidad aritmética} = S2'$$

$$\text{Unidad lógica} = S2$$

Con esta separación, el circuito evita que las salidas aritméticas y lógicas se muestren al mismo tiempo.

Mapas de Karnaugh

Tabla de selección de operaciones

S2	S1	S0	Operación
0	0	0	Suma
0	0	1	Resta
0	1	0	Multiplicación
0	1	1	Potencia
1	0	0	AND
1	0	1	OR
1	1	0	NAND
1	1	1	XNOR

Suma

La suma se activa únicamente cuando S2 = 0, S1 = 0 y S0 = 0.

S2 \ S1S0	00	01	11	10
0	1	0	0	0
1	0	0	0	0

Función simplificada: Suma = $S2' \cdot S1' \cdot S0'$

Resta

La resta se activa cuando S2 = 0, S1 = 0 y S0 = 1.

S2 \ S1S0	00	01	11	10
0	0	1	0	0
1	0	0	0	0

Función simplificada: Resta = $S2' \cdot S1' \cdot S0$

Multiplicación

La multiplicación se activa cuando S2 = 0, S1 = 1 y S0 = 0.

S2 \ S1S0	00	01	11	10
0	0	0	0	1
1	0	0	0	0

Función simplificada: Multiplicación = $S2' \cdot S1 \cdot S0'$

Potencia

La potencia se activa cuando $S2 = 0$, $S1 = 1$ y $S0 = 1$.

$S2 \setminus S1S0$	00	01	11	10
0	0	0	1	0
1	0	0	0	0

Función simplificada: Potencia = $S2' \cdot S1 \cdot S0$

AND

La operación AND se activa cuando $S2 = 1$, $S1 = 0$ y $S0 = 0$.

$S2 \setminus S1S0$	00	01	11	10
0	0	0	0	0
1	1	0	0	0

Función simplificada: AND = $S2 \cdot S1' \cdot S0'$

OR

La operación OR se activa cuando $S2 = 1$, $S1 = 0$ y $S0 = 1$.

$S2 \setminus S1S0$	00	01	11	10
0	0	0	0	0
1	0	1	0	0

Función simplificada: OR = $S2 \cdot S1' \cdot S0$

NAND

La operación NAND se activa cuando $S2 = 1$, $S1 = 1$ y $S0 = 0$.

$S2 \setminus S1S0$	00	01	11	10
0	0	0	0	0
1	0	0	0	1

Función simplificada: $NAND = S2 \cdot S1 \cdot S0'$

XNOR

La operación XNOR se activa cuando $S2 = 1$, $S1 = 1$ y $S0 = 1$.

S2 \ S1S0	00	01	11	10
0	0	0	0	0
1	0	0	1	0

Función simplificada: $XNOR = S2 \cdot S1 \cdot S0$

Unidad Aritmética

La unidad aritmética se activa cuando $S2 = 0$, porque las primeras cuatro combinaciones corresponden a suma, resta, multiplicación y potencia.

S2 \ S1S0	00	01	11	10
0	1	1	1	1
1	0	0	0	0

Función simplificada: Unidad aritmética = $S2'$

Unidad Lógica

La unidad lógica se activa cuando $S2 = 1$, porque las últimas cuatro combinaciones corresponden a AND, OR, NAND y XNOR.

S2 \ S1S0	00	01	11	10
0	0	0	0	0
1	1	1	1	1

Función simplificada: Unidad lógica = $S2$

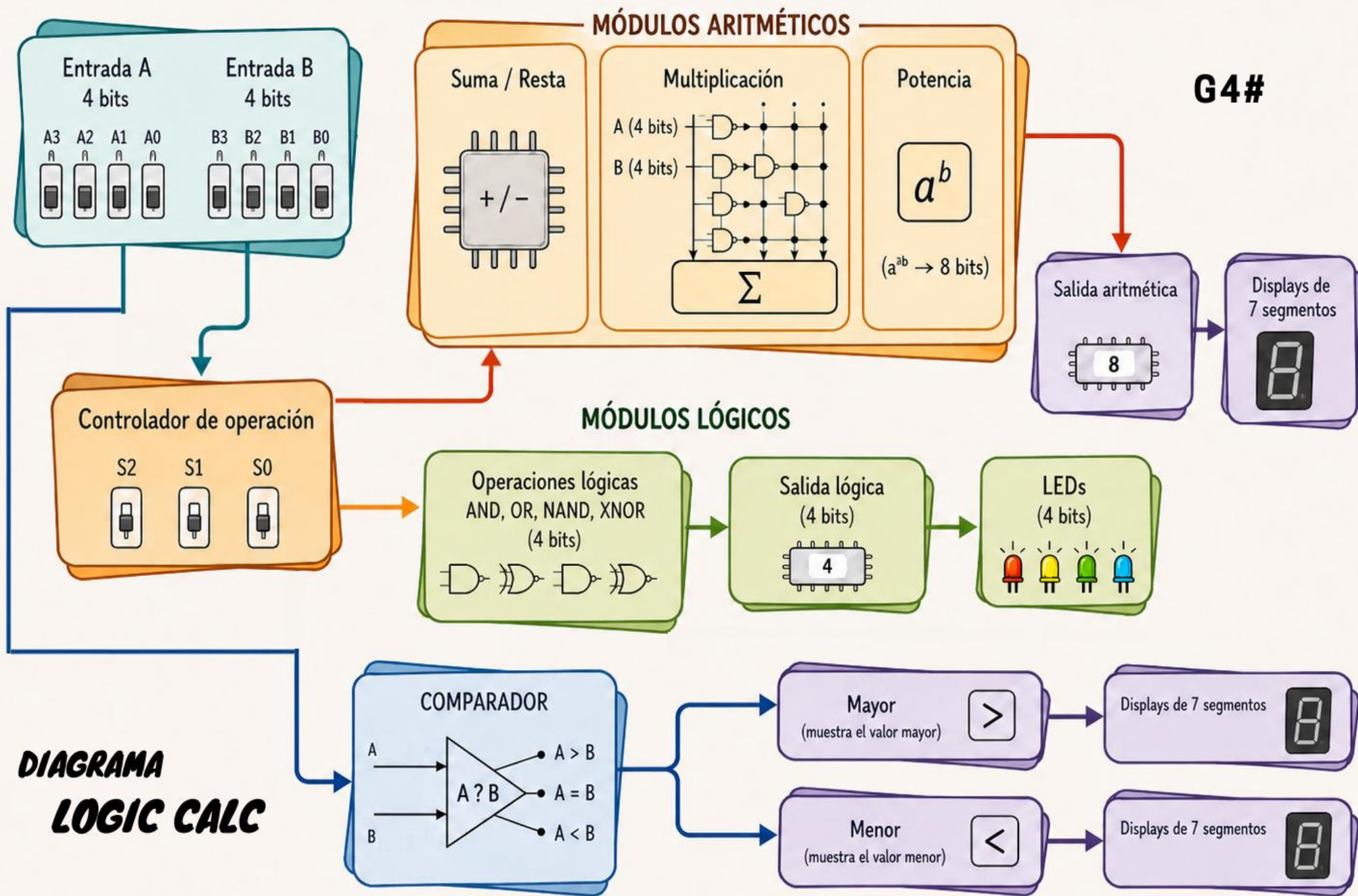


Diagrama del Circuito Físico

Multiplificador!



Decenas

Unidades

Materiales Utilizados:

Para la construcción y simulación de esta práctica se utilizaron los siguientes componentes electrónicos:

Material / Componente	Código o referencia
Compuertas AND	7408
Compuertas OR	7432
Compuertas NOT	7404
Compuertas XOR	7486
Sumadores binarios	7483 / 74283
Comparador binario	7485 / 74285
Multiplexores	74157
Demultiplexor / Decodificador	74138
Decodificador para display	7447 / 7448
Displays de 7 segmentos	Ánodo común / cátodo común
LEDs	5 mm
Resistencias	220 Ω / 330 Ω
Transistores NPN	2N2222
Protoboard	—
Cables	—
Fuente de alimentación	—
Software de simulación	Proteus

Presupuesto:

Material	Cantidad comprada	Precio unitario	Subtotal
DIP SWITCH 3 posiciones tipo integrado	1	Q4.00	Q4.00
DIP SWITCH 1 posición tipo integrado	2	Q3.00	Q6.00
Compuerta lógica NOT 74LS04	6	Q5.00	Q30.00
Compuerta lógica AND 74LS08	20	Q5.00	Q100.00
Compuerta lógica XOR 74LS86	2	Q6.00	Q12.00
Compuerta lógica OR 74LS32	7	Q5.00	Q35.00
Decoder / Demux 74LS138	1	Q6.00	Q6.00
Decodificador demultiplexor 74LS154	2	Q35.00	Q70.00
Sumador de 4 bits 74LS283	6	Q14.00	Q84.00
Sumador 74LS83	2	Q15.00	Q30.00
Comparador 74LS85	2	Q11.00	Q22.00
Multiplexor 2 entradas 74LS157	8	Q7.00	Q56.00
Decodificador/controlador BCD a 7 segmentos 74LS47	5	Q11.00	Q55.00
Display de 7 segmentos ánodo común rojo	2	Q5.00	Q10.00
Cables para conexiones	1	Q40.00	Q40.00
Resistencias para LEDs y displays	20	Q1.25	Q25.00
LEDs	10	Q2.00	Q20.00
Protoboard	2	Q30.00	Q60.00
Subtotal de materiales			Q665.00
Envío			Q35.00
Total general			Q700.00

Nota: Se aclara que este presupuesto solo responde a materiales comprados, dado que los materiales faltantes para realizar esta práctica fueron dados por diversos integrantes del grupo, quienes ya contaban con estos mismos.

Roles del Equipo:

Para el desarrollo de la práctica, el trabajo se distribuyó de la siguiente forma: Esteban Ramos estuvo encargado de las operaciones de suma y resta. Adrián Coronado desarrolló el módulo de multiplicación y Randy Lone trabajó en la unidad de potencia. José Salguero estuvo encargado de la unidad lógica, donde se implementaron las operaciones AND, OR, NAND y XNOR. Angie Godoy y Kristhoper Ucelo trabajaron en la unidad comparativa y en la elaboración de la documentación de la práctica.

Aunque las actividades se dividieron por módulos, todo el proyecto se realizó de forma grupal, cada integrante tuvo una parte asignada, pero todos colaboraron cuando alguien necesitaba ayuda y al momento de unir el circuito completo.

Conclusiones:

General:

El desarrollo de LogicCalc permitió aplicar los conceptos de lógica digital y lógica combinacional en un circuito funcional, integrando diferentes módulos como suma, resta, multiplicación, potencia, operaciones lógicas y comparación de magnitudes. Esto ayudó a comprender de forma práctica cómo una ALU básica puede procesar información binaria mediante compuertas e integrados TTL.

La simulación en Proteus fue una parte importante de esta práctica, ya que permitió verificar el comportamiento de los módulos antes de realizar el montaje físico. Gracias a esta etapa se pudieron identificar errores de conexión, revisar las señales de salida y comprobar que cada bloque respondiera correctamente según la operación seleccionada.

Finalmente, el trabajo en equipo fue fundamental para completar la práctica, ya que cada etapa requirió coordinación entre el diseño, la simulación, el montaje y la documentación. Esta práctica permitió reforzar la importancia de organizar correctamente los módulos de un circuito digital para lograr un sistema funcional y entendible.

Individual:

Jose Daniel Salguero Saldivar: Esta práctica permitió aplicar conceptos fundamentales de lógica digital para transformar señales de entrada en respuestas específicas dentro del sistema. A través de las operaciones lógicas y el uso del demultiplexor como controlador, se evidenció cómo los circuitos combinacionales pueden organizar, seleccionar y dirigir información de manera estructurada para cumplir una función determinada.

Angie Renata Godoy Portillo: En la parte del comparador se verificó el funcionamiento de las entradas A y B para determinar cuándo un número era mayor, menor o igual. También se comprobó que las salidas del comparador podían usarse para controlar la selección de datos en el circuito. Durante la conexión se notó que era importante mantener las entradas bien definidas, ya que un cable mal conectado o una señal flotante podía provocar resultados incorrectos. En general, esta etapa permitió entender mejor cómo el comparador ayuda a tomar decisiones dentro de un circuito.

Adrián Obed Coronado Bueso: Para diseñar el circuito multiplicador, se aprovecharon las ventajas del sistema binario, donde multiplicar bit a bit solo da como resultado 0 o 1, lo que permitió usar compuertas AND (74LS08) para obtener los productos iniciales. Después, estos resultados se acumularon utilizando sumadores de 4 bits en cascada (74LS283); en esta etapa se descartó el octavo bit por redundancia, ya que el diseño está limitado al número 100 y ese bit por sí solo vale 128. Finalmente, para mostrar el resultado en los dos displays de 7 segmentos, se utilizó el algoritmo Double Dabble para convertir el binario puro a BCD. Este método desplaza los bits hacia la izquierda y suma 3 unidades cada vez que un registro es mayor o igual a 5, lo que evita desbordes al duplicar los valores y permite dividir el número limpiamente en decenas y unidades para activar cada pantalla.

Eric Esteban Ramos Sagastume: Yo realicé la parte del sumador y restador de la ALU. Para esto utilicé el integrado 74LS83 como sumador de 4 bits y compuertas XOR 74LS86 para poder hacer la resta mediante complemento a dos. Al principio me costó entender por qué la resta se hacía como una suma, pero después comprendí que cuando la señal SUB está en 0, el circuito suma normalmente, y cuando está en 1, invierte el número B y le suma 1 para obtener la resta.

También agregué el comparador 74LS85 para detectar cuando el número A es menor que B en una resta, ya que en ese caso el resultado sería negativo y se debe mostrar error en los displays. Esta parte me pareció algo complicada por la cantidad de conexiones y porque hay que cuidar mucho el orden de los bits, pero me ayudó a entender mejor cómo una ALU puede usar los mismos componentes para realizar diferentes operaciones.

Randy Xavier Lone Peñate: Se determinó que el diseño de sistemas combinacionales complejos mediante matrices de decodificación presenta una alta vulnerabilidad al ruido eléctrico y a las caídas de tensión por resistencia de contacto en la protoboard. El despliegue incorrecto de caracteres no respondió a una falla lógica del diagrama, sino a un acoplamiento erróneo de impedancias o cables sueltos que dejaron las entradas del decodificador 74LS47 en estado flotante. Esto subraya que en la ingeniería de hardware, la integridad física de las conexiones es tan crítica como la validez del álgebra booleana aplicada.

Kristhoper Franklin Ucelo Pirir: Interesante práctica, me permitió comprender mejor cómo se integran los distintos módulos de una ALU mediante lógica combinacional. Durante la práctica aprendí a hacer operaciones comparativas utilizando

compuertas, ver conexiones, corregir errores y comprobar los resultados en displays y LEDs. En general, la práctica me ayudó a relacionar la teoría con el funcionamiento real del circuito.